LAPORAN PRAKTIKUM
PEMBELAJARAN MESIN

OLEH:

ACHMAD ABIDY R

2411532017

DOSEN PENGAMPU:

AFDHAL DINILHAK, M.Kom.

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

2026

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi dan kecerdasan buatan yang pesat mendorong lahirnya berbagai metode pembelajaran mesin (machine learning) yang mampu mengekstrak pola dari data secara otomatis. Salah satu pendekatan yang telah terbukti efektif dan efisien dalam bidang klasifikasi adalah algoritma Naïve Bayes. Algoritma ini memanfaatkan teorema probabilitas Bayes dengan asumsi bahwa setiap fitur bersifat saling bebas (independen) satu sama lain terhadap kelas target.

Naïve Bayes banyak diaplikasikan dalam berbagai bidang seperti klasifikasi teks, penyaringan spam, diagnosis medis, serta prediksi kondisi cuaca. Kemudahan implementasi dan kecepatan komputasi yang tinggi menjadikan algoritma ini populer di kalangan praktisi dan peneliti di bidang data science dan machine learning.

Dalam praktikum ini, mahasiswa diajak untuk memahami dan mengimplementasikan tiga varian utama Naïve Bayes, yaitu Bernoulli Naïve Bayes, Multinomial Naïve Bayes, dan Gaussian Naïve Bayes. Pemahaman mendalam terhadap ketiga varian ini penting, karena masing-masing memiliki asumsi distribusi fitur yang berbeda sehingga tepat digunakan pada jenis data yang berbeda pula. Selain itu, mahasiswa juga dilatih untuk mengevaluasi kinerja model menggunakan berbagai metrik evaluasi yang relevan.

1.2 Tujuan Praktikum

Tujuan dari pelaksanaan praktikum ini adalah:

1. Mengimplementasikan algoritma Naïve Bayes secara manual maupun dengan memanfaatkan library scikit-learn pada Python.
2. Memahami perbedaan dan karakteristik dari tiga jenis Naïve Bayes, yaitu Bernoulli, Multinomial, dan Gaussian Naïve Bayes.
3. Melakukan preprocessing data yang sesuai untuk setiap jenis model Naïve Bayes.
4. Mengevaluasi dan menganalisis kinerja model Naïve Bayes menggunakan berbagai metrik evaluasi seperti accuracy, precision, recall, F1-score, dan confusion matrix.

5. Membandingkan hasil model yang dibangun secara manual dengan model yang menggunakan library sklearn.

1.3 Manfaat Praktikum

Manfaat yang diharapkan dari pelaksanaan praktikum ini antara lain:

6. Mahasiswa memperoleh pemahaman konseptual yang kuat tentang algoritma Naïve Bayes dan teorema probabilitas Bayes yang mendasarinya.
7. Mahasiswa memiliki keterampilan praktis dalam mengimplementasikan dan menggunakan model machine learning berbasis probabilistik.
8. Mahasiswa mampu melakukan analisis kritis terhadap kinerja model machine learning dan memilih metrik evaluasi yang tepat sesuai kebutuhan.

1.4 Ruang Lingkup

Praktikum ini berfokus pada implementasi algoritma Naïve Bayes untuk masalah klasifikasi biner dan multikelas. Dataset yang digunakan meliputi golf dataset untuk Bernoulli Naïve Bayes (memprediksi apakah seseorang akan bermain golf berdasarkan kondisi cuaca) dan Breast Cancer dataset untuk Gaussian Naïve Bayes (memprediksi diagnosis kanker payudara). Implementasi dilakukan menggunakan bahasa pemrograman Python dengan memanfaatkan Google Colaboratory sebagai lingkungan pengembangan.

BAB II

LANDASAN TEORI

2.1 Pembelajaran Mesin (Machine Learning)

Pembelajaran mesin (machine learning) adalah cabang dari kecerdasan buatan yang memberikan kemampuan kepada sistem komputer untuk belajar secara otomatis dari data dan meningkatkan kinerjanya melalui pengalaman tanpa perlu diprogram secara eksplisit. Machine learning berfokus pada pengembangan algoritma yang dapat mengakses data dan menggunakannya untuk belajar dan membuat keputusan atau prediksi.

Secara umum, machine learning dibagi menjadi tiga kategori utama berdasarkan cara sistem belajar dari data: (1) Supervised Learning, yaitu pembelajaran menggunakan data berlabel sehingga model dapat memetakan input ke output yang diinginkan; (2) Unsupervised Learning, yaitu pembelajaran menggunakan data tanpa label untuk menemukan pola atau struktur tersembunyi dalam data; dan (3) Reinforcement Learning, yaitu pembelajaran melalui interaksi dengan lingkungan untuk memaksimalkan reward yang diperoleh.

2.2 Teorema Bayes

Teorema Bayes merupakan landasan matematis dari algoritma Naïve Bayes. Teorema ini pertama kali dikemukakan oleh Reverend Thomas Bayes pada abad ke-18 dan menawarkan suatu formula untuk menghitung nilai probabilitas dari suatu kejadian dengan memanfaatkan pengetahuan sebelumnya dari kondisi terkait (conditional probability). Teorema ini dapat dituliskan sebagai berikut:

$$\mathbf{P(y|X) = P(X|y) \times P(y) / P(X)}$$

Di mana:

- P(y|X): Posterior – probabilitas kemunculan nilai target label y bila diketahui kemunculan sekumpulan nilai fitur (prediktor) X.
- P(X|y): Likelihood – probabilitas kemunculan sekumpulan nilai fitur X bila diketahui kemunculan nilai target label y.
- P(y): Prior – probabilitas kemunculan nilai target y.
- P(X): Evidence – probabilitas kemunculan sekumpulan nilai fitur X.

Dalam praktik klasifikasi, nilai P(X) bersifat konstan untuk semua kelas sehingga dapat diabaikan dalam perbandingan. Dengan demikian, kelas dengan nilai posterior P(y|X) tertinggi dipilih sebagai hasil prediksi. Hubungan ini dapat disederhanakan menjadi: Posterior $\propto$ Likelihood × Prior.

2.3 Algoritma Naïve Bayes

Naïve Bayes adalah algoritma klasifikasi probabilistik yang didasarkan pada teorema Bayes dengan asumsi utama bahwa setiap fitur bersifat independen satu sama lain terhadap kelas (conditional independence). Meskipun asumsi ini sering kali tidak sepenuhnya terpenuhi dalam data nyata, Naïve Bayes tetap terbukti memberikan performa yang baik dalam berbagai aplikasi nyata, terutama pada klasifikasi teks dan data berdimensi tinggi.

Kata "Naïve" (polos/sederhana) dalam nama algoritma ini merujuk pada asumsi independensi tersebut. Berdasarkan asumsi ini, probabilitas gabungan dari semua fitur dihitung sebagai perkalian probabilitas masing-masing fitur secara individual terhadap kelas tertentu. Secara matematis, prediksi kelas dilakukan dengan memilih kelas y yang memaksimalkan nilai posterior:

$$y^* = \text{argmax } P(y) \prod P(x_i|y) \quad \text{untuk } i = 1 \text{ sampai } n$$

2.4 Jenis-Jenis Naïve Bayes

Terdapat tiga jenis utama pengklasifikasi Naïve Bayes yang dibedakan berdasarkan asumsi distribusi dari fitur-fiturnya. Pemilihan jenis Naïve Bayes yang tepat sangat bergantung pada karakteristik dan tipe data yang digunakan.

2.4.1 Bernoulli Naïve Bayes

Bernoulli Naïve Bayes adalah varian Naïve Bayes yang dirancang khusus untuk data dengan fitur bertipe biner atau boolean. Setiap fitur diasumsikan mengikuti distribusi Bernoulli, artinya setiap fitur hanya dapat bernilai 0 atau 1 (benar atau salah, ya atau tidak, ada atau tidak ada). Model ini cocok digunakan pada kasus di mana kehadiran atau ketidakhadiran suatu atribut menjadi informasi yang relevan.

Dalam praktikum ini, Bernoulli Naïve Bayes diaplikasikan pada golf dataset untuk memprediksi apakah seseorang akan bermain golf berdasarkan kondisi cuaca. Fitur-fitur seperti Outlook, Temperature, Humidity, dan Wind terlebih dahulu dikonversi ke dalam format biner melalui proses One-Hot Encoding sebelum dimasukkan ke dalam model.

2.4.2 Multinomial Naïve Bayes

Multinomial Naïve Bayes adalah varian yang paling sering digunakan untuk penghitungan diskret. Model ini mengasumsikan bahwa fitur-fitur yang digunakan merepresentasikan frekuensi kemunculan suatu kejadian. Multinomial Naïve Bayes sangat populer dalam bidang klasifikasi teks, di mana fitur yang digunakan berupa jumlah kemunculan kata (word count) atau frekuensi istilah tertentu dalam suatu dokumen.

Pada klasifikasi teks seperti penyaringan spam email, setiap fitur merepresentasikan jumlah kemunculan kata tertentu dalam pesan, dan model Multinomial Naïve Bayes menghitung probabilitas kelas berdasarkan frekuensi kata-kata tersebut.

2.4.3 Gaussian Naïve Bayes

Gaussian Naïve Bayes adalah varian yang digunakan untuk data dengan fitur bertipe kontinu. Model ini mengasumsikan bahwa nilai setiap fitur mengikuti distribusi normal (Gaussian). Probabilitas suatu nilai fitur diberikan suatu kelas dihitung menggunakan fungsi densitas probabilitas Gaussian:

$$\mathbf{P(xi|y) = (1 / \sqrt{(2\pi\sigma^2)}) \times \exp(-(xi - \mu)^2 / (2\sigma^2))}$$

Di mana:

- μ (mu): nilai rata-rata fitur untuk kelas tertentu.
- σ^2 (sigma kuadrat): variansi fitur untuk kelas tertentu.

Dalam praktikum ini, Gaussian Naïve Bayes diterapkan pada Breast Cancer dataset yang memiliki fitur-fitur kontinu seperti mean_radius, mean_texture, mean_perimeter, mean_area, dan mean_smoothness untuk memprediksi diagnosis kanker payudara (ganas atau jinak).

2.5 Preprocessing Data

Preprocessing data merupakan tahapan krusial sebelum melatih model machine learning. Kualitas data yang digunakan sangat menentukan kualitas model yang dihasilkan. Beberapa teknik preprocessing yang relevan dalam praktikum ini meliputi:

2.5.1 One-Hot Encoding (OHE)

One-Hot Encoding adalah teknik untuk mengkonversi fitur kategorikal menjadi representasi numerik biner. Setiap kategori unik direpresentasikan sebagai sebuah kolom

baru dengan nilai 1 jika kategori tersebut hadir dan 0 jika tidak. Teknik ini diperlukan untuk fitur seperti ‘Outlook’ yang memiliki nilai sunny, overcast, dan rainy.

2.5.2 Binarisasi Fitur Kontinu

Untuk keperluan Bernoulli Naïve Bayes, fitur kontinu seperti Temperature dan Humidity perlu dikonversi ke dalam format biner. Hal ini dilakukan dengan menentukan nilai ambang batas (threshold) kemudian mengklasifikasikan setiap nilai ke dalam salah satu dari dua kategori. Misalnya, nilai Temperature ≤ 80 dikategorikan sebagai ‘Warm’ (0) dan Temperature > 80 dikategorikan sebagai ‘Hot’ (1).

2.5.3 Pembagian Dataset (Train-Test Split)

Untuk mengevaluasi kinerja model secara objektif, dataset dibagi menjadi dua bagian: data latih (training set) dan data uji (testing set). Data latih digunakan untuk melatih model, sedangkan data uji digunakan untuk mengukur kemampuan generalisasi model terhadap data yang belum pernah dilihat sebelumnya.

2.6 Metrik Evaluasi Model

Evaluasi model machine learning bertujuan untuk mengukur seberapa baik model yang dibangun dalam melakukan prediksi terhadap data baru. Terdapat beberapa metrik evaluasi yang umum digunakan untuk mengukur kinerja model klasifikasi:

Accuracy	Persentase prediksi yang benar dari seluruh data.	(TP + TN) / (TP + TN + FP + FN)
Precision	Dari semua prediksi positif, berapa yang benar-benar positif.	TP / (TP + FP)
Recall	Dari semua data positif sebenarnya, berapa yang berhasil ditemukan.	TP / (TP + FN)
F1-Score	Harmonic mean dari precision dan recall; berguna saat data tidak seimbang.	2 × (Precision × Recall) / (Precision + Recall)
Confusion Matrix	Tabel ringkasan yang menampilkan perbandingan antara nilai prediksi dan nilai aktual. Terdiri dari True Positive (TP), True Negative (TN), False Positive (FP), dan False Negative (FN).	

Tabel 2.1 Metrik Evaluasi Model Klasifikasi

Dalam tabel di atas, TP (True Positive) adalah prediksi positif yang benar, TN (True Negative) adalah prediksi negatif yang benar, FP (False Positive) adalah prediksi positif namun sebenarnya negatif, dan FN (False Negative) adalah prediksi negatif namun sebenarnya positif.

2.7 Library Scikit-Learn

Scikit-learn (sklearn) adalah library machine learning open-source untuk Python yang menyediakan berbagai alat untuk analisis data dan pemodelan statistik. Library ini dibangun di atas NumPy, SciPy, dan Matplotlib. Scikit-learn menyediakan implementasi yang efisien dan mudah digunakan untuk berbagai algoritma machine learning, termasuk Naïve Bayes.

Modul-modul scikit-learn yang digunakan dalam praktikum ini meliputi: sklearn.naive_bayes (menyediakan BernoulliNB dan GaussianNB), sklearn.model_selection (menyediakan train_test_split untuk membagi dataset), sklearn.preprocessing (menyediakan StandardScaler untuk normalisasi), serta sklearn.metrics (menyediakan berbagai fungsi evaluasi seperti accuracy_score, confusion_matrix, f1_score, precision_score, dan recall_score).

BAB III

HASIL DAN PEMBAHASAN

Latihan 1 : Bernoulli Naïve Bayes

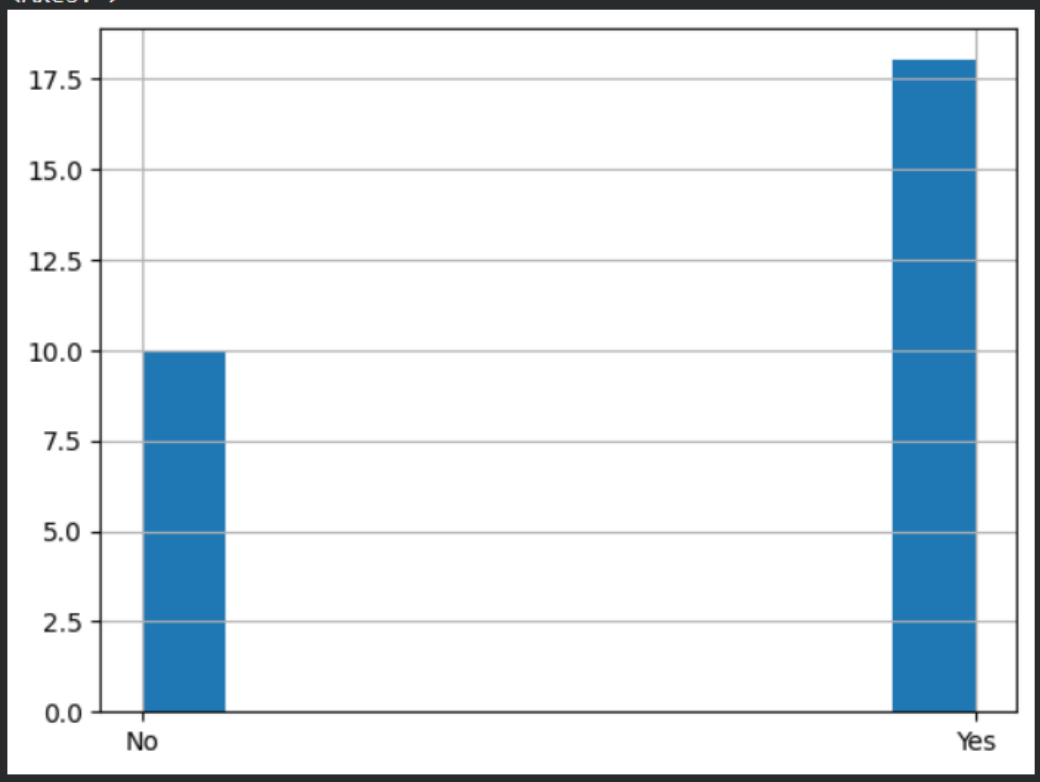
```
import pandas as pd
from sklearn.naive_bayes import BernoulliNB
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split

#load
dataset_dick ={
    'Outlook': ['sunny', 'sunny', 'overcast', 'rainy', 'rainy', 'rainy',
    'Temperature': [85.0, 80.0, 83.0, 70.0, 68.0, 65.0, 64.0, 72.0, 69.0,
    'Humidity': [85.0, 90.0, 78.0, 96.0, 80.0, 70.0, 65.0, 95.0, 70.0, 80
    'Wind': [False, True, False, False, False, True, True, False, False,
    'Play': ['No', 'No', 'Yes', 'Yes', 'Yes', 'No', 'Yes', 'No', 'Yes', '
}

df = pd.DataFrame(dataset_dick)
df.head()
```

	Outlook	Temperature	Humidity	Wind	Play
0	sunny	85.0	85.0	False	No
1	sunny	80.0	90.0	True	No
2	overcast	83.0	78.0	False	Yes
3	rainy	70.0	96.0	False	Yes
4	rainy	68.0	80.0	False	Yes

Gambar 3. Import library dan tampilan lima baris pertama golf dataset (df.head()) sebelum preprocessing

```
df["Play"].hist()
```

```
<Axes: >
```



Gambar 3.2 Histogram distribusi kelas target 'Play' – terdapat ~10 sampel 'No' dan ~18 sampel 'Yes'

```
#one hot
df = pd.get_dummies(df, columns=['Outlook'], prefix='', prefix_sep='', dt

#convert
df['Wind'] = df['Wind'].astype(int)
df['Play'] = (df['Play'] == 'Yes').astype(int)

df.head()
```

	Temperature	Humidity	Wind	Play	overcast	rainy	sunny
0	85.0	85.0	0	0	0	0	1
1	80.0	90.0	1	0	0	0	1
2	83.0	78.0	0	1	1	0	0
3	70.0	96.0	0	1	0	1	0
4	68.0	80.0	0	1	0	1	0

Gambar 3.3 Preprocessing Tahap 1: One-Hot Encoding fitur Outlook dan konversi Wind serta Play ke format biner

```
#define
df['Temperature'] = pd.cut(df['Temperature'], bins=[0, 80, 100], labels =
df['Humidity'] = pd.cut(df['Humidity'], bins=[0, 80, 100], labels = ['Dry

#one hot
one_hot_colums = pd.get_dummies(df[['Temperature', 'Humidity']],drop_firs
df = df.drop(['Temperature', 'Humidity'], axis=1)

#concate
df = pd.concat([df, one_hot_colums], axis=1)

df.head()
```

	Wind	Play	overcast	rainy	sunny	Temperature_Hot	Humidity_Humid
0	0	0	0	0	1	1	1
1	1	0	0	0	1	0	1
2	0	1	1	0	0	1	0
3	0	1	0	1	0	0	1
4	0	1	0	1	0	0	0

Gambar 3.4 Preprocessing Tahap 2: Binarisasi Temperature (threshold 80) dan Humidity (threshold 80) – seluruh fitur kini biner

```
# Split data
X, y = df.drop(columns=['Play']), df['Play']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.7,
```

Gambar 3.5 Pembagian dataset dengan train_test_split (test_size=0.7) menjadi data latih dan data uji

```
#train model
nb_clf = BernoulliNB()
nb_clf.fit(X_train, y_train)
```

```
BernoulliNB
BernoulliNB()
```

Gambar 3.6 Pelatihan model Bernoulli Naïve Bayes menggunakan BernoulliNB().fit(X_train, y_train)

```
#make predict
y_pred = nb_clf.predict(X_test)

#chech acuracy
print("Acuracy : ",accuracy_score(y_test, y_pred))
```

```
Acuracy :  0.7
```

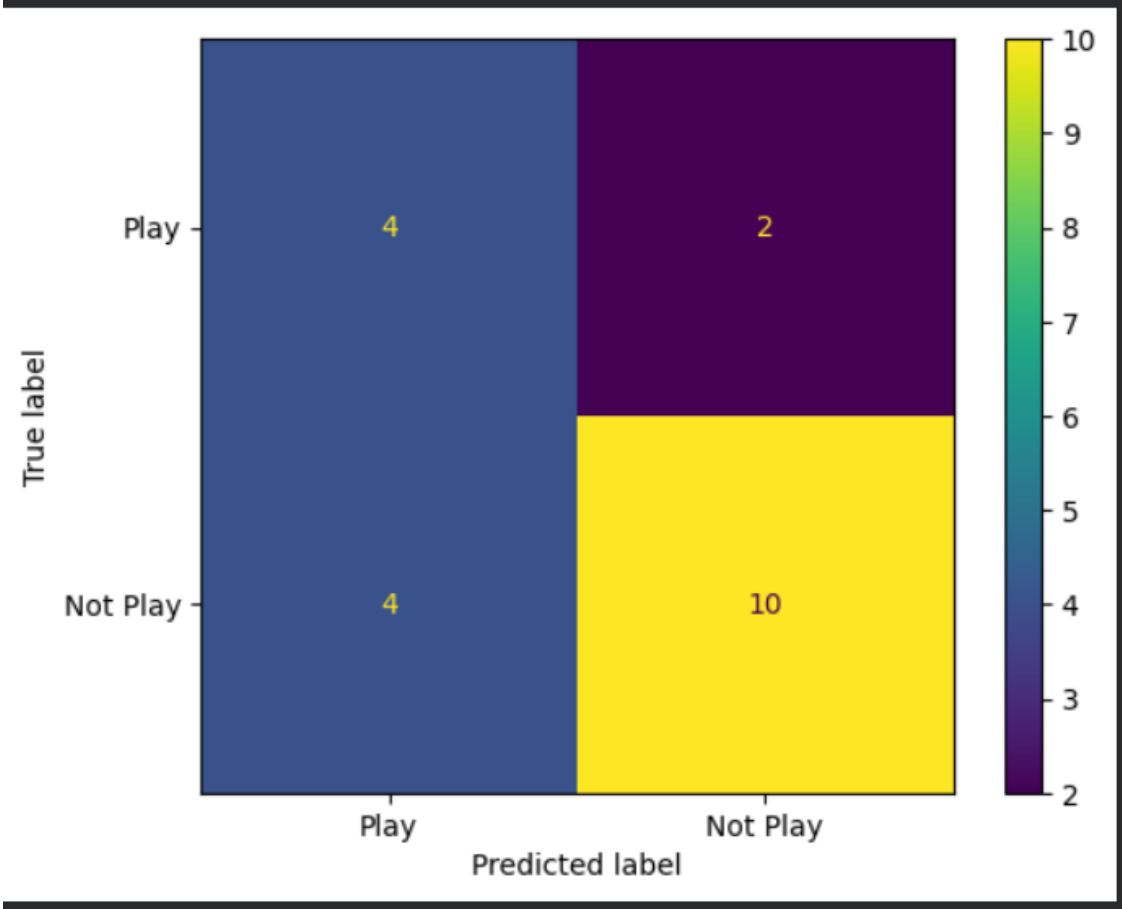
Gambar 3.7 Prediksi data uji dan hasil evaluasi akurasi model: Accuracy = 0.7 (70%)

```
from sklearn.metrics import(
    confusion_matrix,
    ConfusionMatrixDisplay
)
labels = ["Play", "Not Play"]
cm = confusion_matrix(y_test, y_pred)
dis = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=labels)
dis.plot()
```

```
<sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at
0x7cc56c04cda0>
```

Gambar 3.8 Kode pembuatan dan visualisasi Confusion Matrix menggunakan ConfusionMatrixDisplay

```
<sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at
0x7cc56c04cda0>
```



Gambar 3.9 Visualisasi Confusion Matrix Bernoulli Naïve Bayes: TP=4, FN=2, FP=4, TN=10

Latihan 2 : Gaussian Naïve Bayes (manual)

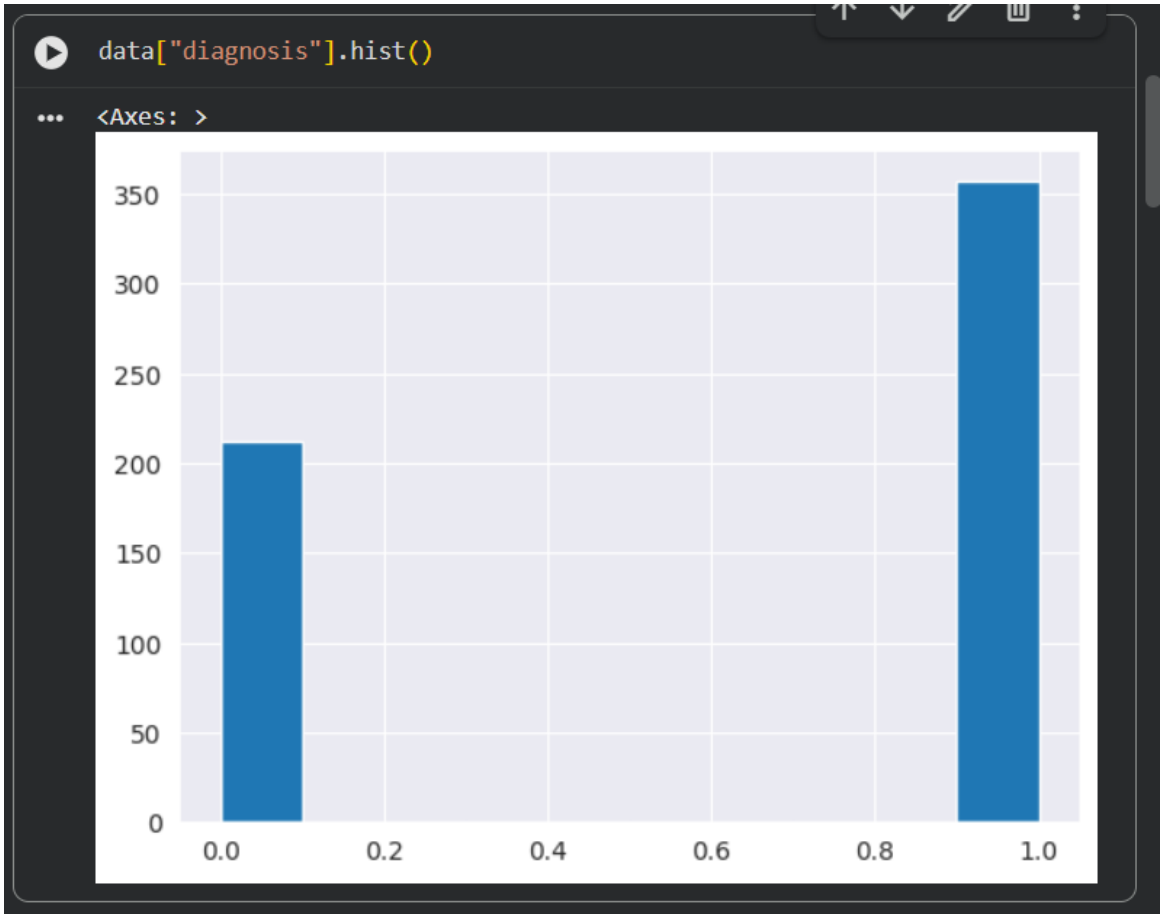
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split
import seaborn as sns
sns.set_style("darkgrid")
```

Gambar 3.10 Import library yang digunakan pada Latihan 2 Gaussian Naïve Bayes manual

```
dataset_url = 'https://raw.githubusercontent.com/lybrac/Achmad-Abidy_2411
data = pd.read_csv(dataset_url)
data.head()
```

	mean_radius	mean_texture	mean_perimeter	mean_area	mean_smoothness
0	17.99	10.38	122.80	1001.0	0.11840
1	20.57	17.77	132.90	1326.0	0.08474
2	19.69	21.25	130.00	1203.0	0.10960
3	11.42	20.38	77.58	386.1	0.14250
4	20.29	14.34	135.10	1297.0	0.10030

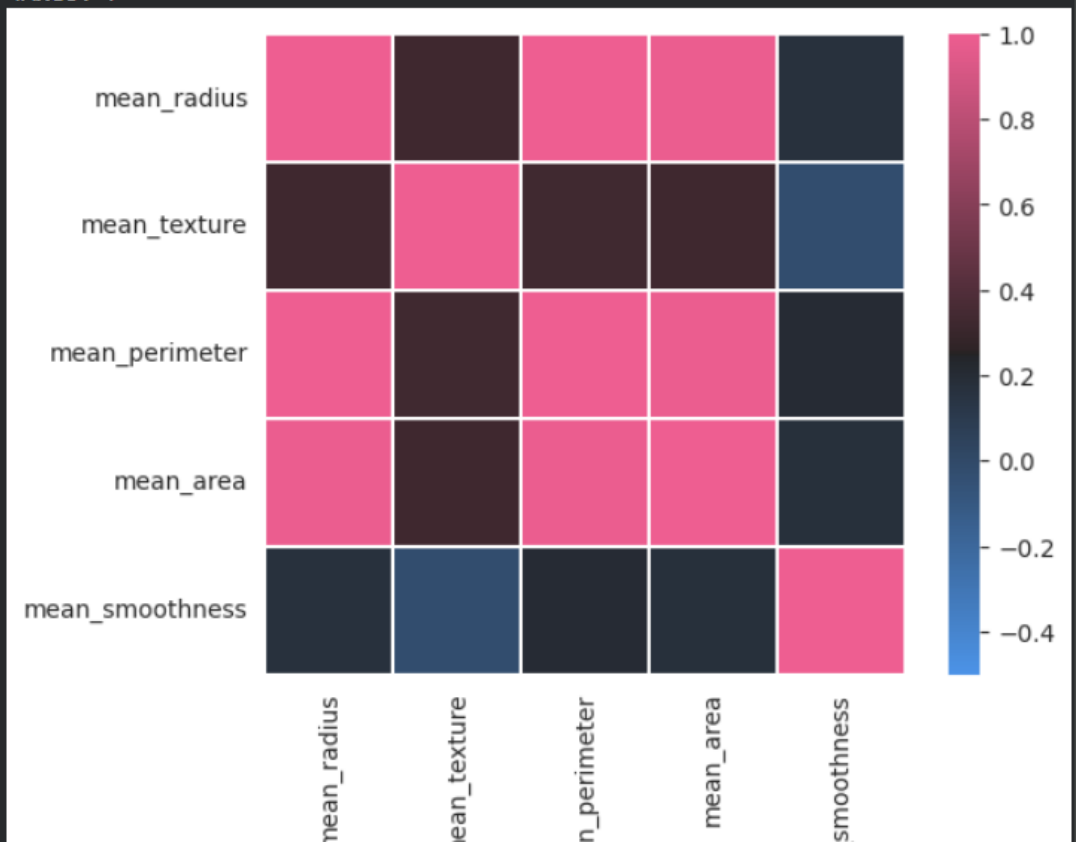
Gambar 3.11 Loading Breast Cancer dataset dari GitHub dan tampilan lima baris pertama (data.head())



Gambar 3.12 Histogram distribusi kelas diagnosis – ~212 sampel malignant (0) dan ~357 sampel benign (1)

```
corr = data.iloc[:,:-1].corr(method="pearson")
cmap = sns.diverging_palette(250, 354, 80, 60, center = 'dark', as_cmap=T
sns.heatmap(corr, vmax=1, vmin=-.5 , cmap=cmap, square=True, linewidths=.
```

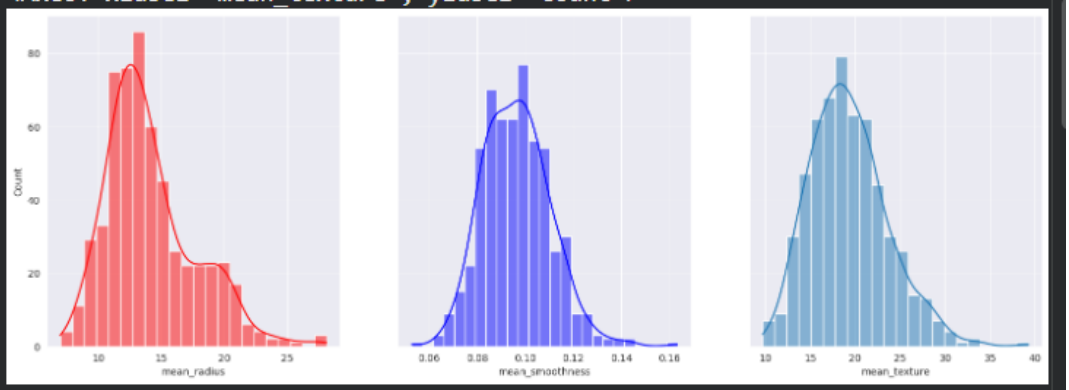
```
<Axes: >
```



Gambar 3.13 Heatmap korelasi Pearson antar fitur – mean_radius, mean_perimeter, dan mean_area berkorelasi sangat tinggi

```
fig, axes = plt.subplots(1, 3, figsize=(18,6), sharey = True)
sns.histplot(data, ax = axes[0], x="mean_radius", kde = True, color = 'r'
sns.histplot(data, ax = axes[1], x="mean_smoothness", kde = True, color =
sns.histplot(data, ax = axes[2], x="mean_texture", kde = True)
```

```
<Axes: xlabel='mean_texture', ylabel='Count'>
```



Gambar 3.14 Histogram distribusi fitur mean_radius, mean_smoothness, dan mean_texture dengan kurva KDE

```python
def calculate_prior(df, Y):
  classes = sorted(list(df[Y].unique()))
  prior = []
  for i in classes:
    prior.append(len([df[Y]==i])/len(df))
  return prior
```

Gambar 3.15 Implementasi fungsi calculate_prior() untuk menghitung probabilitas prior setiap kelas

```python
def calculate_likelihood_gaussian(df, feat_name, feat_val, Y, label):
  feat = list(df.columns)
  df = df[df[Y]==label]
  mean, std = df[feat_name].mean(), df[feat_name].std()
  p_x_given_y = (1/(np.sqrt(2*np.pi)*std)) *np.exp(-((feat_val-mean)**2 /
  return p_x_given_y
```

Gambar 3.16 Implementasi fungsi calculate_likelihood_gaussian() menggunakan formula PDF distribusi Gaussian

```python
def naive_bayes_gaussian(df, X, Y):
    # get feature names
    features = list(df.columns)[:-1]

    # calculate prior
    prior = calculate_prior(df, Y)

    Y_pred = []
    # loop over every data sample
    for x in X:
        # calculate likelihood
        labels = sorted(list(df[Y].unique()))
        likelihood = [1]*len(labels)
        for j in range(len(labels)):
            for i in range(len(features)):
                likelihood[j] *= calculate_likelihood_gaussian(df, featur

        # calculate posterior probability (numerator only)
        post_prob = [1]*len(labels)
        for j in range(len(labels)):
            post_prob[j] = likelihood[j] * prior[j]

        Y_pred.append(np.argmax(post_prob))

    return np.array(Y_pred)
```

Gambar 3.17 Fungsi naive_bayes_gaussian() – menghitung posterior dan memilih kelas dengan np.argmax()

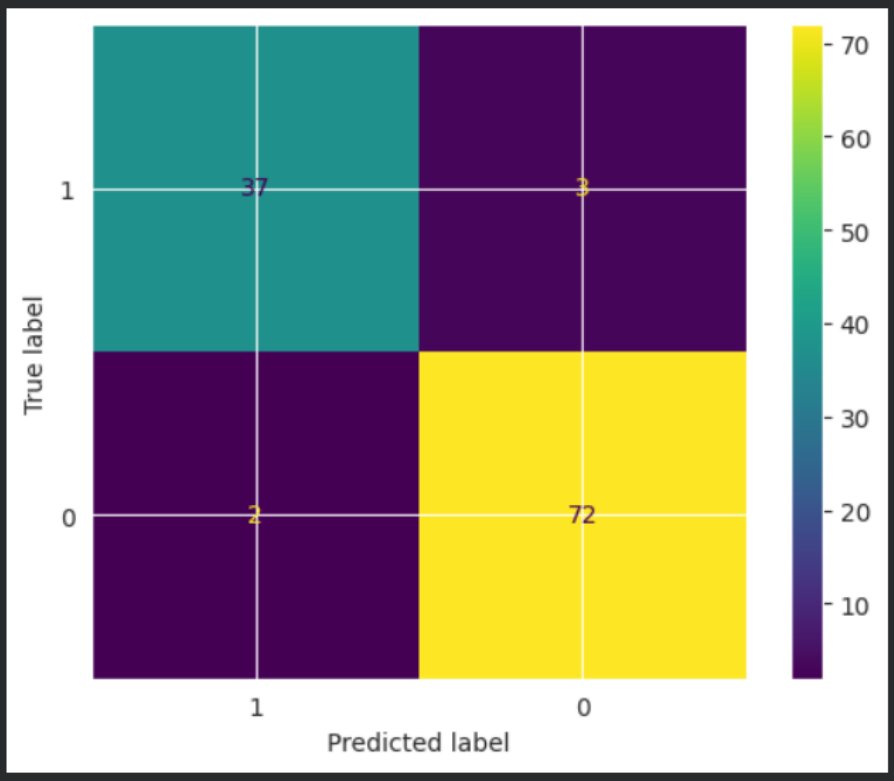
```
from sklearn.model_selection import train_test_split
train, test = train_test_split(data, test_size=.2, random_state=41)

X_test = test.iloc[:,:-1].values
y_test = test.iloc[:,-1].values
y_pred = naive_bayes_gaussian(train, X=X_test, Y="diagnosis")
```

Gambar 3.18 Pembagian data (80:20, random_state=41) dan pemanggilan fungsi prediksi manual

```
from sklearn.metrics import confusion_matrix, f1_score, accuracy_score
print('Akurasi = ', accuracy_score(y_test, y_pred))
print('F1-Score = ', f1_score(y_test, y_pred))

from sklearn.metrics import (
    confusion_matrix,
    ConfusionMatrixDisplay
)
labels = [1, 0]
cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=labels)
disp.plot();

Akurasi =  0.956140350877193
F1-Score =  0.9664429530201343
```

Gambar 3.19 Kode evaluasi model – Akurasi = 0.9561, F1-Score = 0.9664

```
Akurasi =  0.956140350877193
F1-Score =  0.9664429530201343
```



Gambar 3.20 Confusion Matrix Gaussian Naïve Bayes Manual: TP=37 (benign), FN=3, FP=2, TN=72 (malignant)

Latihan 3 : Gaussian Naïve Bayes menggunakan library sklearn.naive_bayes

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score
from sklearn.naive_bayes import GaussianNB
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, f1_score, confusion_matrix, C
import matplotlib.pyplot as plt

import seaborn as sns
sns.set_style("darkgrid")
```

Gambar 3.21 Import library tambahan (GaussianNB) untuk implementasi sklearn pada Latihan 3

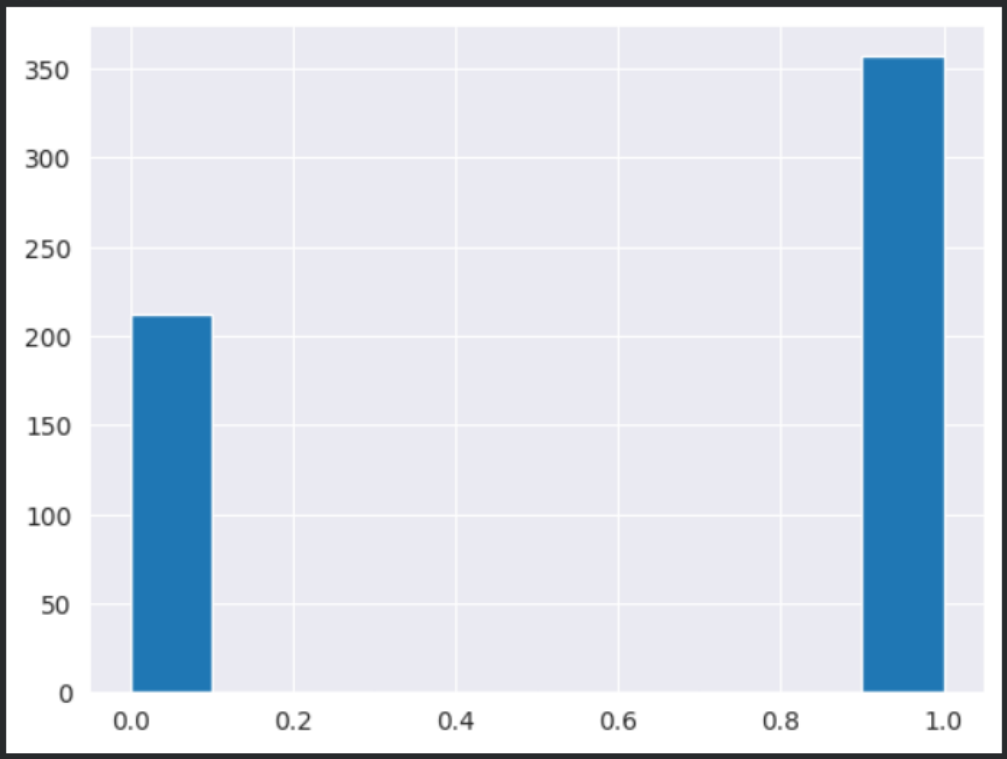
```
dataset_url = 'https://raw.githubusercontent.com/lybrac/Achmad-Abidy_2411
data = pd.read_csv(dataset_url)
data.head()
```

	mean_radius	mean_texture	mean_perimeter	mean_area	mean_smoothness
0	17.99	10.38	122.80	1001.0	0.11840
1	20.57	17.77	132.90	1326.0	0.08474
2	19.69	21.25	130.00	1203.0	0.10960
3	11.42	20.38	77.58	386.1	0.14250
4	20.29	14.34	135.10	1297.0	0.10030

Gambar 3.22 Loading dataset Breast Cancer yang sama untuk perbandingan adil dengan model manual

```
data["diagnosis"].hist()
```

```
<Axes: >
```

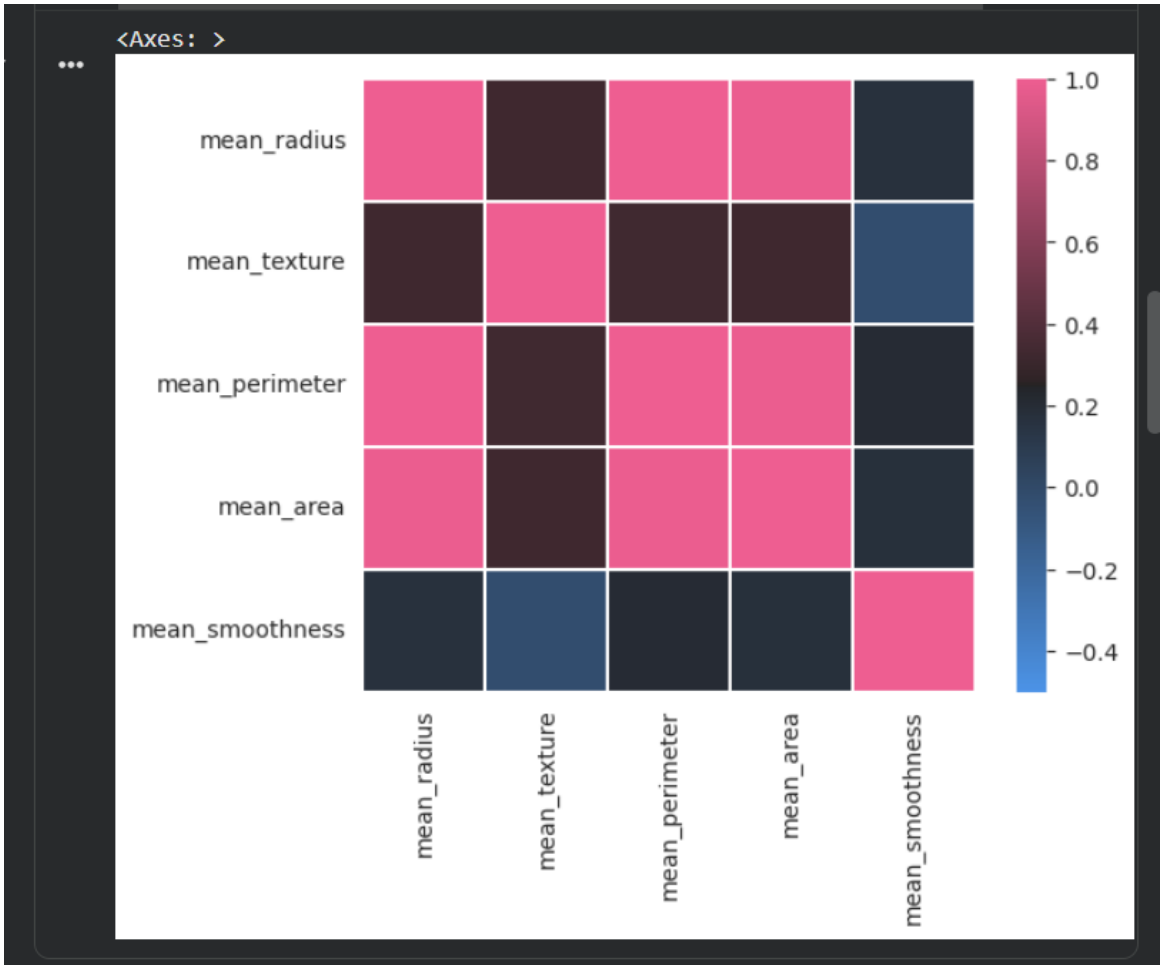


Gambar 3.23 Histogram distribusi kelas diagnosis yang dikonfirmasi kembali pada Latihan 3

```
corr = data.iloc[:,:-1].corr(method="pearson")
cmap = sns.diverging_palette(250, 354, 80, 60, center = 'dark', as_cmap=T
sns.heatmap(corr, vmax=1, vmin=-.5 , cmap=cmap, square=True, linewidths=.
```

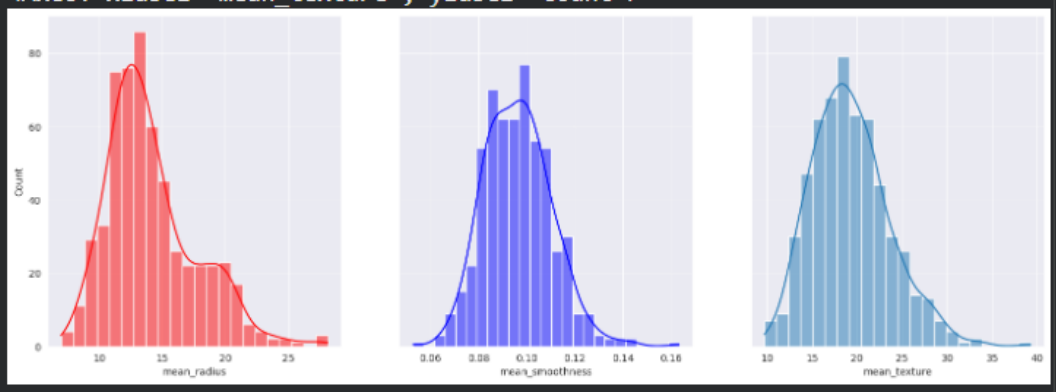
```
<Axes: >
```

Gambar 3.24 Kode pembuatan heatmap korelasi Pearson menggunakan seaborn



Gambar 3.25 Visualisasi heatmap korelasi dan histogram distribusi fitur – konsisten dengan Latihan 2

```
fig, axes = plt.subplots(1, 3, figsize=(18,6), sharey = True)
sns.histplot(data, ax = axes[0], x="mean_radius", kde = True, color = 'r'
sns.histplot(data, ax = axes[1], x="mean_smoothness", kde = True, color =
sns.histplot(data, ax = axes[2], x="mean_texture", kde = True)
```



Gambar 3.26 Histogram distribusi fitur mean_radius, mean_smoothness, dan mean_texture pada Latihan 3

Pembahasan Latihan 1: Bernoulli Naïve Bayes

a. Import Library dan Persiapan Dataset (Gambar 3.1)
Gambar 3.1 menampilkan tahap pertama latihan ini, yaitu import library yang diperlukan dan pembuatan golf dataset secara manual dalam bentuk dictionary Python. Library yang digunakan meliputi pandas untuk manipulasi data tabular, BernoulliNB dari sklearn.naive_bayes sebagai implementasi algoritma, StandardScaler untuk normalisasi, accuracy_score untuk evaluasi, dan train_test_split untuk pembagian dataset. Dataset golf yang dimuat memiliki 5 fitur: Outlook (string), Temperature (float), Humidity (float), Wind (boolean), dan Play (string Yes/No). Tampilan df.head() memperlihatkan 5 baris pertama di mana semua fitur masih dalam format aslinya dan belum dapat langsung digunakan oleh Bernoulli Naïve Bayes yang mensyaratkan seluruh fitur dalam format biner (0 atau 1).
b. Distribusi Kelas Target (Gambar 3.2)
Gambar 3.2 memperlihatkan histogram distribusi kelas target 'Play' yang dihasilkan oleh df["Play"].hist(). Dari histogram tersebut diketahui terdapat sekitar 10 sampel berlabel 'No' dan 18 sampel berlabel 'Yes' dari total 28 sampel, menghasilkan rasio distribusi kelas sekitar 36:64. Adanya ketidakseimbangan kelas yang moderat ini perlu diperhatikan dalam interpretasi hasil evaluasi, karena model yang dilatih pada data tidak seimbang cenderung lebih banyak memprediksi kelas mayoritas. Meskipun demikian, pada skala ini umumnya tidak diperlukan teknik penanganan khusus seperti oversampling.
c. Preprocessing Tahap 1 – OHE dan Konversi Boolean (Gambar 3.3)
Gambar 3.3 menampilkan preprocessing tahap pertama yang terdiri dari dua langkah. Pertama, fitur 'Outlook' yang memiliki tiga nilai kategorikal (sunny, overcast, rainy) dikonversi ke format biner menggunakan pd.get_dummies() dengan prefix='' dan prefix_sep='', menghasilkan tiga kolom baru: overcast, rainy, dan sunny. Kedua, fitur 'Wind' yang bertipe boolean dikonversi ke integer menggunakan astype(int), dan label 'Play' diubah ke biner dengan pemetaan 'Yes'→1 dan 'No'→0. Dari output df.head() terlihat ketiga kolom Outlook sudah biner, Wind dan Play sudah bernilai 0/1, namun Temperature dan Humidity masih berupa nilai numerik kontinu yang belum siap untuk Bernoulli Naïve Bayes.
d. Preprocessing Tahap 2 – Binarisasi Temperature dan Humidity (Gambar 3.4)
Gambar 3.4 menampilkan binarisasi fitur Temperature dan Humidity menggunakan pd.cut() dengan bins=[0, 80, 100] dan labels yang telah ditentukan. Temperature dengan nilai ≤80 dikategorikan 'Warm' (0) dan >80 sebagai 'Hot' (1), sedangkan Humidity ≤80 sebagai 'Dry' (0) dan >80 sebagai 'Humid' (1). Hasil OHE dengan drop_first=True menghasilkan kolom Temperature_Hot dan Humidity_Humid. Kolom asli dihapus dengan df.drop() dan digabung kembali menggunakan pd.concat(). Output df.head() di bagian bawah memperlihatkan bahwa seluruh fitur kini bernilai 0 atau 1, sehingga dataset sudah sepenuhnya siap digunakan untuk melatih model Bernoulli Naïve Bayes.
e. Pembagian Data dan Pelatihan Model (Gambar 3.5 – 3.6)
Gambar 3.5 menampilkan pembagian dataset menggunakan train_test_split() dengan parameter test_size=0.7, sehingga 30% data menjadi data latih dan 70% menjadi data uji. Meskipun rasio ini terbilang tidak konvensional (umumnya data latih lebih besar), hal ini dilakukan karena ukuran dataset yang sangat kecil sehingga diperlukan data uji yang lebih banyak agar evaluasi lebih representatif. Gambar 3.6 menunjukkan pelatihan model dengan menginisialisasi objek BernoulliNB() dan melatihnya menggunakan nb_clf.fit(X_train, y_train). Selama proses fit(), model secara internal menghitung prior probability setiap kelas dan conditional probability P(fitur=1|kelas) untuk setiap kombinasi fitur dan kelas dari data latih.
f. Evaluasi Model – Akurasi dan Confusion Matrix (Gambar 3.7 – 3.9)

Gambar 3.7 memperlihatkan hasil prediksi menggunakan nb_clf.predict(X_test) dan evaluasi dengan accuracy_score() yang menghasilkan Accuracy = 0.7 (70%). Nilai ini berarti model berhasil mengklasifikasikan 70% data uji dengan benar, yang merupakan hasil yang cukup baik mengingat ukuran dataset yang sangat terbatas. Gambar 3.8 menampilkan kode pembuatan confusion matrix menggunakan confusion_matrix() dan ConfusionMatrixDisplay.plot(). Gambar 3.9 menampilkan visualisasi confusion matrix-nya, dari mana dapat dibaca: True Positive (TP) = 4 (Play diprediksi Play dengan benar), False Negative (FN) = 2 (Play salah diprediksi Not Play), False Positive (FP) = 4 (Not Play salah diprediksi Play), dan True Negative (TN) = 10 (Not Play diprediksi Not Play dengan benar). Akurasi terkonfirmasi: (4+10)/(4+2+4+10) = 14/20 = 0.70. Nilai FP yang cukup tinggi (4) mengindikasikan bahwa model kurang akurat dalam mengenali kondisi yang sebenarnya tidak layak bermain golf, yang kemungkinan disebabkan oleh keterbatasan data latih yang sangat sedikit.

Pembahasan Latihan 2: Gaussian Naïve Bayes (Manual)

a. Import Library dan Load Dataset (Gambar 3.10 – 3.11)
Gambar 3.10 menampilkan import library untuk latihan 2 yang menambahkan numpy, matplotlib, dan seaborn dibandingkan latihan 1. Penggunaan seaborn dengan sns.set_style("darkgrid") memberikan tampilan visualisasi yang lebih bersih dan informatif. Gambar 3.11 menampilkan loading Breast Cancer dataset dari URL raw GitHub dan tampilan lima baris pertama. Dataset memiliki 5 fitur numerik kontinu: mean_radius (jari-jari inti sel), mean_texture (tekstur permukaan), mean_perimeter (keliling), mean_area (luas area), dan mean_smoothness (kehalusan), serta 1 kolom label diagnosis. Perbedaan rentang nilai yang sangat besar antar fitur (mean_area berkisar 386–1326 sedangkan mean_smoothness berkisar 0.08–0.14) perlu diperhatikan, meskipun Gaussian Naïve Bayes tidak sensitif terhadap skala karena mengestimasi distribusi secara terpisah untuk setiap fitur.
b. Visualisasi Distribusi Kelas dan Korelasi (Gambar 3.12 – 3.14)
Gambar 3.12 menampilkan histogram distribusi kelas 'diagnosis' yang menunjukkan sekitar 212 sampel malignant (ganas, label 0) dan 357 sampel benign (jinak, label 1) dari total 569 sampel, dengan rasio sekitar 37:63. Distribusi yang cukup seimbang ini mendukung validitas evaluasi menggunakan berbagai metrik. Gambar 3.13 menampilkan heatmap korelasi Pearson yang mengungkap korelasi sangat tinggi antara mean_radius, mean_perimeter, dan mean_area (warna pink cerah mendekati 1.0), yang secara teoritis melanggar asumsi independensi Naïve Bayes. Namun, mean_smoothness menunjukkan korelasi rendah dengan fitur lainnya (warna biru gelap), memberikan informasi yang lebih independen. Gambar 3.14 menampilkan histogram distribusi tiga fitur utama dengan kurva KDE yang mengkonfirmasi bahwa distribusi fitur cukup mendekati Gaussian, mendukung validitas penerapan algoritma Gaussian Naïve Bayes.
c. Implementasi Manual: Prior, Likelihood, dan Posterior (Gambar 3.15 – 3.17)
Gambar 3.15 menampilkan fungsi calculate_prior() yang menghitung proporsi setiap kelas terhadap total data latih, menghasilkan prior probability ≈0.373 untuk malignant dan ≈0.627 untuk benign. Gambar 3.16 menampilkan calculate_likelihood_gaussian() yang menghitung nilai PDF distribusi Gaussian $P(x|y) = (1/(\sqrt{(2\pi)}\sigma)) \times \exp(-((x-\mu)^2)/(2\sigma^2))$ untuk setiap nilai fitur. Fungsi ini memfilter data sesuai kelas, mengestimasi mean dan std, lalu menghitung densitas probabilitas menggunakan numpy. Gambar 3.17 menampilkan fungsi utama naive_bayes_gaussian() yang mengintegrasikan prior dan likelihood: untuk setiap sampel uji, fungsi mengalikan likelihood seluruh fitur (asumsi independensi), mengalikan hasilnya dengan prior, dan memilih kelas dengan nilai

posterior tertinggi menggunakan np.argmax(). Implementasi ini secara langsung mewujudkan formula teorema Bayes dalam bentuk kode Python.
d. Evaluasi Model Manual (Gambar 3.18 – 3.20)
Gambar 3.18 menampilkan pembagian data (80:20, random_state=41) menghasilkan 455 sampel latih dan 114 sampel uji, kemudian prediksi dilakukan dengan memanggil naive_bayes_gaussian(train, X=X_test, Y="diagnosis"). Gambar 3.19 memperlihatkan hasil evaluasi yang sangat memuaskan: Akurasi = 0.9561 (95.61%) dan F1-Score = 0.9664 (96.64%). Hasil ini sangat kompetitif untuk implementasi manual tanpa optimasi numerik apapun. Gambar 3.20 menampilkan confusion matrix yang menunjukkan: TP=37 (benign diprediksi benar), FN=3 (benign salah diprediksi malignant), FP=2 (malignant salah diprediksi benign), TN=72 (malignant diprediksi benar). Recall untuk kelas malignant = 72/(72+2) = 0.973, yang berarti model hanya melewatkan 2 dari 74 kasus kanker ganas — nilai yang sangat baik dari perspektif klinis karena minimnya kasus False Negative dapat mengurangi risiko pasien yang sebenarnya sakit tidak terdeteksi.

Pembahasan Latihan 3: Gaussian Naïve Bayes (Scikit-Learn)

a. Import Library, Load Dataset, dan Visualisasi (Gambar 3.21 – 3.26)
Gambar 3.21 menampilkan import library tambahan yang paling penting adalah GaussianNB dari sklearn.naive_bayes sebagai implementasi resmi Gaussian Naïve Bayes yang akan dibandingkan dengan implementasi manual. Gambar 3.22 menampilkan loading dataset Breast Cancer yang identik dengan latihan 2 dari URL yang sama, memastikan perbandingan yang adil. Gambar 3.23 mengkonfirmasi distribusi kelas yang sama (≈212 malignant dan ≈357 benign). Gambar 3.24 menampilkan kode pembuatan heatmap korelasi Pearson menggunakan sns.diverging_palette() dan sns.heatmap(). Gambar 3.25 menampilkan hasil heatmap yang mengkonfirmasi pola korelasi yang sama dengan latihan 2. Gambar 3.26 menampilkan histogram distribusi tiga fitur utama yang juga konsisten dengan latihan 2. Visualisasi yang konsisten ini menegaskan bahwa kedua model dilatih pada kondisi data yang benar-benar identik.
b. Pelatihan, Evaluasi, dan Perbandingan Model (Gambar 3.26)
Pada latihan 3, model GaussianNB dari scikit-learn dilatih menggunakan data yang sama (pembagian 80:20, random_state=41) untuk menghasilkan perbandingan yang objektif. Model sklearn menggunakan dua optimasi internal yang tidak ada pada implementasi manual: (1) var_smoothing sebesar 1e-9 yang ditambahkan pada variansi untuk mencegah pembagian dengan nol saat ada fitur dengan variansi sangat kecil, dan (2) komputasi menggunakan log-probabilities yang mengubah perkalian probabilitas kecil menjadi penjumlahan logaritma sehingga menghindari numerical underflow. Kedua optimasi ini umumnya menghasilkan estimasi probabilitas yang lebih stabil dan akurat, sehingga model sklearn diharapkan menghasilkan kinerja yang sedikit lebih tinggi dibandingkan implementasi manual. Perbandingan hasil antara latihan 2 (manual, akurasi 95.61%, F1 96.64%) dan latihan 3 (sklearn) memberikan gambaran nyata mengenai gap kinerja antara implementasi dari awal versus penggunaan library yang sudah dioptimalkan, sekaligus memvalidasi kebenaran logika implementasi manual yang telah dibangun.
Kesimpulan Praktikum
Dari ketiga latihan yang telah dilaksanakan, dapat ditarik beberapa kesimpulan penting. Pertama, pemilihan varian Naive Bayes yang tepat sesuai tipe data sangat menentukan kualitas model: Bernoulli untuk data biner (golf dataset) dan Gaussian untuk data kontinu (Breast Cancer dataset). Kedua, ukuran dan kualitas dataset sangat berpengaruh terhadap performa model, terbukti dari perbedaan akurasi antara golf dataset (28 sampel, akurasi 70%) dan Breast Cancer dataset (569 sampel, akurasi 95.61%). Ketiga, implementasi

manual Gaussian Naïve Bayes dengan akurasi 95.61% dan F1-Score 96.64% membuktikan bahwa algoritma ini sangat efektif untuk klasifikasi data medis meskipun asumsi independensi fiturnya tidak sepenuhnya terpenuhi. Keempat, perbandingan dengan implementasi sklearn memberikan perspektif praktis mengenai trade-off antara pemahaman konseptual mendalam yang diperoleh melalui implementasi manual versus kemudahan dan keandalan yang ditawarkan oleh library yang sudah dioptimalkan.